

Security_JWT Project에 CI/CD적용하기

시나리오

첫번째,

Mysql은 AWS EC2에 Docker기반으로 docker-compose.yml을 이용하여 실행 시킨다.

mysql은 데이터를 보존해야 하므로 자주 down하면 위험하고 DB는 처음 한번만 띄우고 계속 유지하는 게 좋다.

비록 volume을 설정했다라도, 무분별하게 down/up을 반복하면 **잠재적 데이터 손실** 위험이 있기 때문에 잦은 down/up은 권장하지 않으므로 **자동화 CI/CD에서 제외시킨다.**

두번째,

Sprint Boot 애플리케이션은 코드가 자주 바뀌므로 rebuild + restart가 필요하다.

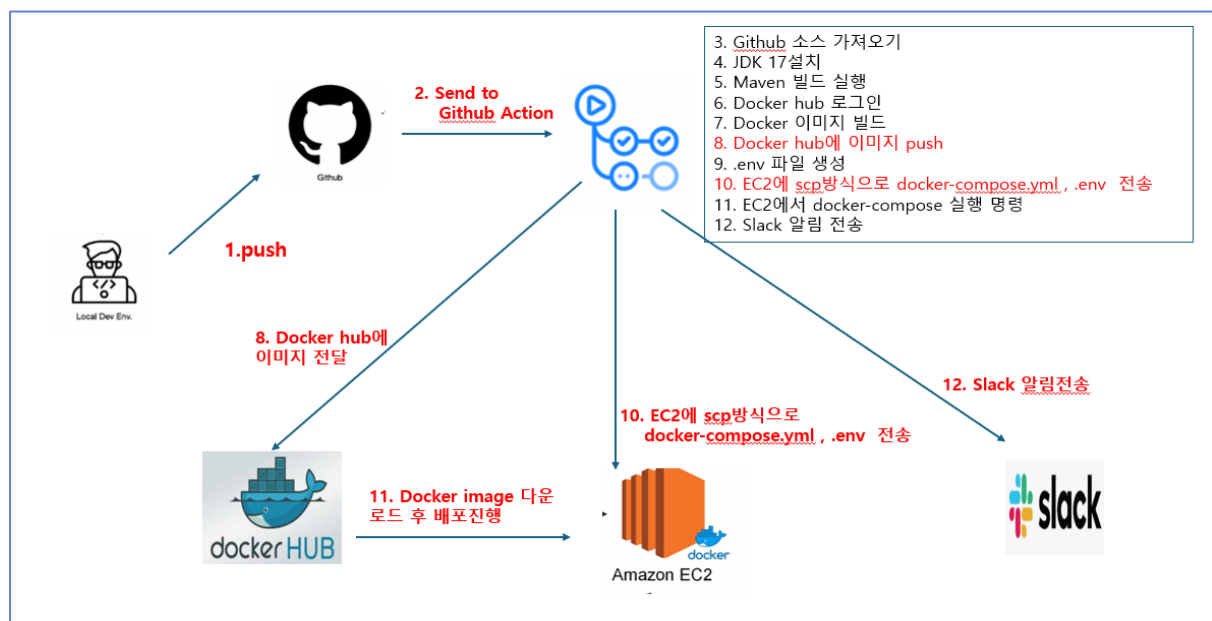
Spring Boot 앱은 코드가 변경될 때마다 jar 파일이 바뀌므로 매번 새로 빌드 된 Docker 이미지로 교체해야 하기 때문에 CI/CD에서는 일반적으로 boot 앱만 docker compose restart로 다시 띄운다.

즉, 코드를 반영하기 위해 boot 서비스만 주기적으로 새로 띄운다.

세번째,

GitHub secrets에 민감한 정보를 저장하고 설정문서에서 사용하도록 한다.

전체흐름도



작업순서

1. EC2생성 (t3.small , port : 8080, 3306)
2. EC2에 Docker 설치
3. mysql 설치 – docker-compose.yml 이용
4. github에 새로운 프로젝트 생성([cicd03_back_security](#)) 하고 boot 프로젝트 연결
 - .gitignore에서 target부분 주석 풀기
 - .gitignore에 application-dev.properties 추가
5. github > Settings > Secrets and Variables > Actions > New repository secret등록
6. boot프로젝트에 설정문서 만들기 및 수정
 - Dockerfile
 - docker-compose.yml
 - .github/workflows/deploy.yml
 - application.properties수정

EC2생성

Amazon EC2를 사용하면 AWS 클라우드에서 실행되는 가장 머신 또는 인스턴스를 생성할 수 있습니다. 아래의 간단한 단계에 따라 빠르게 시작할 수 있습니다.

이름 및 태그 정보

이름

back_boot

추가 태그 추가

▼ 애플리케이션 및 OS 이미지(Amazon Machine Image) 정보

AMI는 인스턴스를 시작하는 데 필요한 소프트웨어 구성(운영 체제, 애플리케이션 서버 및 애플리케이션)이 포함된 템플릿입니다. 아래에서 찾고 있는 항목이 보이지 않으면 AMI를 검색하거나 찾아보세요.

Q 수천 개의 애플리케이션 및 OS 이미지를 포함하는 전체 카탈로그 검색

최근 사용

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Debian

debian

더 많은 AMI 찾아보기

AWS, Marketplace 및 커뮤니티의 AMI 포함

Amazon Machine Image(AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type

ami-0d5bb3742db8fc264 (64비트(x86)) / ami-064b0e4b6529b20cf (64비트(Arm))

가상화: hvm ENA 활성화됨: true 루트 디바이스 유형: ebs

프리 티어 사용 가능

▼ 인스턴스 유형 정보 | 조언 받기

인스턴스 유형

t3.small
제일자리: t3 2 vCPU 2 GiB 메모리 현재 세대: true 온디맨드 Ubuntu Pro 기본 요금: 0.0295 USD 시간당
온디맨드 RHEL 기본 요금: 0.0548 USD 시간당 온디맨드 Linux 기본 요금: 0.026 USD 시간당
온디맨드 Windows 기본 요금: 0.0444 USD 시간당 온디맨드 SUSE 기본 요금: 0.057 USD 시간당

모든 세대
인스턴스 유형 비교

소프트웨어가 사전 설치된 AMI에는 추가 비용이 적용됩니다.

▼ 키 페어(로그인) 정보

키 페어를 사용하여 인스턴스에 안전하게 연결할 수 있습니다. 인스턴스를 시작하기 전에 선택한 키 페어에 대한 액세스 권한이 있는지 확인하세요.

키 페어 이름 - 필수

my-key

새 키 페어 생성

▼ 네트워크 설정 정보

네트워크 | 정보

vpc-029bf3f33d1054705

서브넷 | 정보

편집

▼ 네트워크 설정 정보

네트워크 | 정보

vpc-029bf3f33d1054705

서브넷 | 정보

기본 설정 없음(가용 영역의 기본 서브넷)

퍼블릭 IP 자동 할당 | 정보

활성화

프리 티어 허용 범위를 벗어나는 경우 추가 요금이 적용됩니다.

방화벽(보안 그룹) | 정보

보안 그룹은 인스턴스에 대한 트래픽을 제어하는 방화벽 규칙 세트입니다. 특정 트래픽이 인스턴스에 도달하도록 허용하는 규칙을 추가합니다.

보안 그룹 생성

기존 보안 그룹 선택

다음 규칙을 사용하여 'launch-wizard-1'(이)라는 새 보안 그룹을 생성합니다.

☒ 다음에서 SSH 트래픽 허용
인스턴스 연결에 도움

위치 무관
0.0.0.0/0

☐ 인터넷에서 HTTPS 트래픽 허용
예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

☐ 인터넷에서 HTTP 트래픽 허용
예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

편집

방화벽(보안 그룹) | 정보

보안 그룹은 인스턴스에 대한 트래픽을 제어하는 방화벽 규칙 세트입니다. 특정 트래픽이 인스턴스에 도달하도록 허용하는 규칙을 추가합니다.

보안 그룹 생성

기존 보안 그룹 선택

보안 그룹 이름 - 필수

sec-back

이 보안 그룹은 모든 네트워크 인터페이스에 추가됩니다. 보안 그룹을 만든 후에는 이름을 편집할 수 없습니다. 최대 길이는 255자입니다. 유효한 문자는 a~z, A~Z, 0~9, 공백 및 .-:/()#,@[]+=&:{}!\$*입니다.

설명 - 필수 | 정보

launch-wizard-1 created 2025-07-08T03:35:09.386Z

인바운드 보안 그룹 규칙

▼ 보안 그룹 규칙 1 (TCP, 22, 0.0.0.0/0)

유형 | 정보

ssh

프로토콜 | 정보

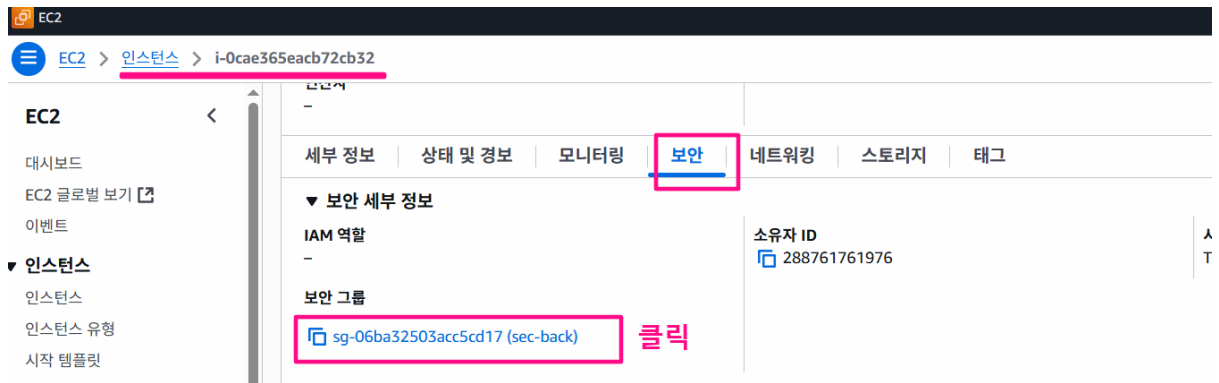
TCP

포트 범위 | 정보

22

제거

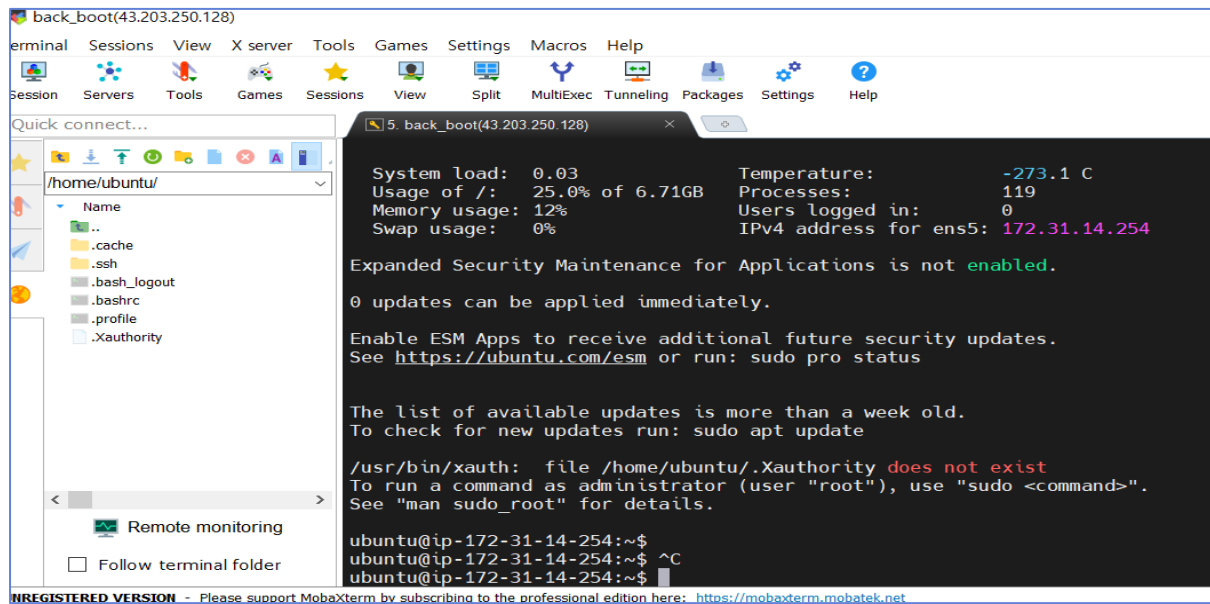
인스턴스 생성 후 > 인스턴스 클릭 > 보안그룹 선택한다.



인바운드 규칙 편집을 클릭하여 규칙을 추가한다(22, 80, 443, 8080, 3306)



MobaXterm접속



Docker 설치

```

sudo apt-get update && ₩
sudo apt-get install -y apt-transport-https ca-certificates curl software-properties-
common && ₩
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add - && ₩
sudo apt-key fingerprint 0EBFCD88 && ₩
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu
$(lsb_release -cs) stable" && ₩
sudo apt-get update && ₩
sudo apt-get install -y docker-ce && ₩
sudo usermod -aG docker ubuntu && ₩
sudo curl -L "https://github.com/docker/compose/releases/download/1.23.2/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose && ₩
sudo chmod +x /usr/local/bin/docker-compose && ₩
sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose
  
```

```

100 11.2M 100 11.2M 0 0 9751k 0 0:00:01 0:00:01 --:--:-- 211M
ubuntu@ip-172-31-14-254:~$ docker --version
Docker version 28.1.1, build 4eba377
ubuntu@ip-172-31-14-254:~$
  
```

Docker를 이용한 Mysql설치

```

#네트워크생성하기
sudo docker network create mynet
  
```

볼륨생성하기

```
sudo docker volume create mydata
```

#EC2에 docker_mysql 폴더생성

```
sudo mkdir docker_mysql
```

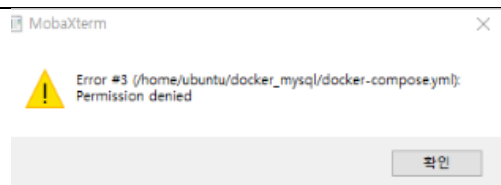
폴더이동

```
cd docker_mysql
```

mobaxterm의 SFTP를 이용하여 제공 파일 업로드

제공한 ex06_springSecurity_mysql > docker_mysql >
docker-compose.yml, init.sql 파일 드래그드랍

드래그앤드랍할 때 Permission denied 발생하면 아래 명령어 실행한다.



```
sudo chown ubuntu:ubuntu /home/ubuntu/docker_mysql
```

ls로 확인해보기

```
ubuntu@ip-172-31-45-106:~/docker_mysql$ ls
docker-compose.yml  init.sql
ubuntu@ip-172-31-45-106:~/docker_mysql$
```

도커컴포즈 실행

```
sudo docker compose up -d
```

도커컴포즈 종료

```
sudo docker compose down
```

확인하기

```
sudo docker ps -a
```

mysql안으로 들어가기

```
sudo docker exec -it mysql_db /bin/bash
```

mysql 접속하기

mysql -ujang -pjang1234

데이터베이스 확인하기

show databases;

#데이터베이스 선택

use jwt_db

Github에 cicd03_back_security 프로젝트를 생성한다.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk ().*

Owner *


8253jang

Repository name *

cicd03_back_security


cicd03_back_security is available.

Great repository names are short and memorable. Need inspiration? How about [literate-succotash](#) ?

Description (optional)

cicd03_back_security > Settings > Secrets and variables > Actions 선택 해서
아래의 Secret 등록한다.

GitHub Secrets 목록(10개)

이름	설명
EC2_HOST	EC2 퍼블릭 IP
EC2_USERNAME	ubuntu
EC2_PRIVATE_KEY	.pem 파일의 텍스트
DOCKER_USERNAME	Docker Hub 사용자명
DOCKER_ACCESS_TOKEN	Docker Hub 액세스 토큰
SPRING_JWT_SECRET	spring.jwt.secret 설정값 SahL5G83mGzrulJzffw-eD7RqphsXluJPBwxu0ncBFo
MYSQL_URL	jdbc:mysql://ec2_public_ip:3306/jwt_db?characterEncoding=UTF-8

MYSQL_DATABASE	jwt_db
MYSQL_USER	jang
MYSQL_PASSWORD	jang1234

Repository secrets

New repository secret

Name 	Last updated
 DOCKER_ACCESS_TOKEN	19 minutes ago  
 DOCKER_USERNAME	4 days ago  
 EC2_HOST	4 days ago  
 EC2_PRIVATE_KEY	4 days ago  
 EC2_USERNAME	4 days ago  
 MYSQL_DATABASE	4 days ago  
 MYSQL_PASSWORD	4 days ago  
 MYSQL_URL	10 hours ago  
 MYSQL_USER	4 days ago  
 SPRING_JWT_SECRET	4 days ago  

제공한 back_security_final 프로젝트를 인텔리제이로 import 하고 github에 push한다.

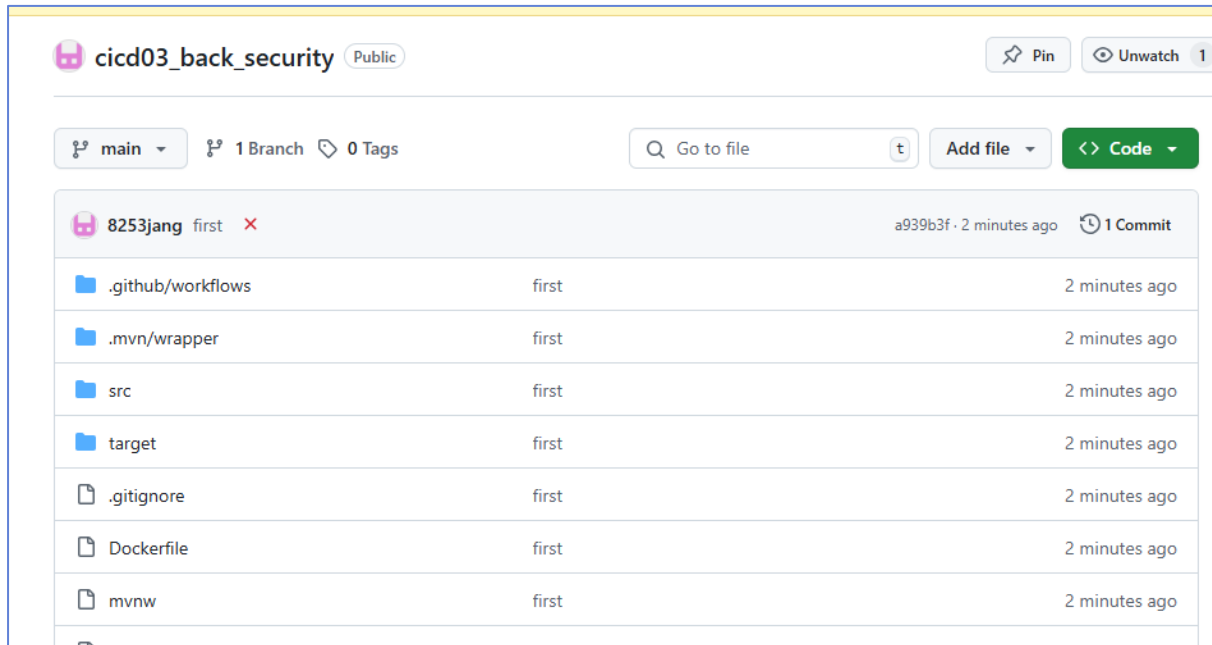
```
KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/back_security
$ git init
Initialized empty Git repository in D:/Edu_286/AWS/back_security/.git

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/back_security (main)
$ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF
it touches it
warning: in the working copy of '.mvn/wrapper/maven-wrapper.properties',
LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'mvnw', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'mvnw.cmd', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'pom.xml', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'src/main/java/web/mvc/Step03SecurityCon
troller.java', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/test/java/web/mvc/Step03SecurityCon
trollerTest.java', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'target/surefire-reports/TEST-web.m
vc.SecurityControllerTests.xml', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'target/surefire-reports/TEST-web.m
vc.SecurityControllerTests.xml', LF will be replaced by CRLF the next time Git
touches it

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/back_security (main)
$ git commit -m "first"
[main (root-commit) a939b3f] first
 80 files changed, 2132 insertions(+)
 create mode 100644 .github/workflows/deploy.yml
 create mode 100644 .gitignore
 create mode 100644 .mvn/wrapper/maven-wrapper.properties
```

```
KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/back_security (main)
$ git remote add origin https://github.com/8253jang/cicd03_back_security.git

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/back_security (main)
$ git push origin main
Enumerating objects: 129, done.
Counting objects: 100% (129/129), done.
Delta compression using up to 8 threads
Compressing objects: 100% (111/111), done.
Writing objects: 100% (129/129), 44.64 MiB | 6.91 MiB/s, done.
Total 129 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (9/9), done.
To https://github.com/8253jang/cicd03_back_security.git
 * [new branch]      main -> main
```



Dockerfile 작성

```

1  # 1. OpenJDK 17 기반의 슬림 이미지 사용 (필수 최소 구성만 포함)
2  >> FROM eclipse-temurin:17-jdk
3
4  # 2. 컨테이너 내에서 앱이 실행될 디렉토리 지정
5  WORKDIR /app
6
7  # 3. 로컬의 빌드 결과물 JAR 파일을 컨테이너에 복사
8  #   - target 디렉토리 내의 특정 JAR 파일을 명시적으로 복사
9  #   - 이름이 변경되면 이 부분도 함께 수정해주는 방식이 안정적
10 COPY target/*-SNAPSHOT.jar app.jar
11
12 # 4. 애플리케이션 실행
13 #   - Asia/Seoul 시간대 설정
14 #   - UTF-8 문자 인코딩 설정 (한글 깨짐 방지)
15 ENTRYPOINT ["java", "-Duser.timezone=Asia/Seoul", "-Dfile.encoding=UTF-8", "-jar", "app.jar"]
16

```

docker-compose.yml 작성

```

1  version: '3' # Docker Compose 파일의 버전. v3은 대부분의 기능을 지원하며 프로덕션 환경에서도 자주 사용됨.
2
3  services:
4    springboot-app: # 서비스 이름 (컨테이너 역할 명시)
5      image: ${DOCKER_USERNAME}/jwt-app:latest # 사용할 도커 이미지. GitHub Actions에서 푸시한 최신 버전 사용
6      container_name: jwt_app # 실제 생성될 컨테이너의 이름 지정
7
8      ports:
9        - "8080:8080" # 호스트의 8080 포트를 컨테이너의 8080 포트에 매핑 (외부에서 접속 가능)
10
11     environment: # 컨테이너에 전달할 환경변수 설정
12       SPRING_JWT_SECRET: ${SPRING_JWT_SECRET} # JWT 시크릿 키 (보안용)
13       MYSQL_URL: ${MYSQL_URL} # DB 접속 URL
14       MYSQL_USER: ${MYSQL_USER} # DB 사용자명
15       MYSQL_PASSWORD: ${MYSQL_PASSWORD} # DB 비밀번호
16       JAVA_TOOL_OPTIONS: "-Duser.timezone=Asia/Seoul -Dfile.encoding=UTF-8" # JVM 시간대/인코딩 설정
17       SPRING_PROFILES_ACTIVE: prod # Spring Boot의 프로파일을 prod(운영)로 설정
18
19     networks:
20       - mynet # 동일한 네트워크에 연결된 다른 컨테이너들과 통신 가능
21
22   networks:
23     mynet:
24       external: true # 이미 생성된 외부 네트워크(mynet 등)를 사용. docker network create mynet 필요
25

```

application.properties에 이미 설정했는데 docker-compose.yml에서
왜 또 environment: 설정이 필요한가?

properties

```

spring.datasource.url=${MYSQL_URL}
spring.datasource.username=${MYSQL_USER}
spring.datasource.password=${MYSQL_PASSWORD}

```

이런 설정은 스프링 부트가 실행될때,
OS 환경변수에서 해당 값을 가져와서
사용한다는 뜻

MYSQL_URL, MYSQL_USER, MYSQL_PASSWORD라는 **운영체제 환경변수**가 실행 환경에
세팅되어 있어야 그 값을 사용할 수 있다.

그런데 도커에서는?

Docker 컨테이너 안에서는 외부에서 전달해주지 않으면 기본적으로 환경변수가 없다.
그래서 docker-compose.yml에서 아래처럼 명시해줘야 한다.

```
environment: # 컨테이너에 전달할 환경변수 설정
  SPRING_JWT_SECRET: ${SPRING_JWT_SECRET} # JWT 시크릿 키 (보안용)
  MYSQL_URL: ${MYSQL_URL} # DB 접속 URL
  MYSQL_USER: ${MYSQL_USER} # DB 사용자명
  MYSQL_PASSWORD: ${MYSQL_PASSWORD} # DB 비밀번호
  JAVA_TOOL_OPTIONS: "-Duser.timezone=Asia/Seoul -Dfile.encoding=UTF-8" # JVM 시간대/인코딩 설정
  SPRING_PROFILES_ACTIVE: prod # Spring Boot의 프로파일을 prod(운영)로 설정
```

application.properties – 공통

```
1  spring.application.name=step03_Security_JWT
2
3  spring.profiles.active=dev
4
5  spring.jwt.secret=${SPRING_JWT_SECRET}
6
7  spring.mvc.pathmatch.matching-strategy=ant_path_matcher
8
9  server.port=8080
10
11  spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
12  spring.jpa.database=mysql
13  spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
14  spring.jpa.generate-ddl=false
15  spring.jpa.show-sql=true
16  spring.jpa.properties.hibernate.format_sql=true
17
```

application-dev.properties – 개발용

```
2  spring.datasource.url=jdbc:mysql://43.203.250.128:3306/jwt_db?characterEncoding=UTF-8
3  spring.datasource.username=jang
4  spring.datasource.password=jang1234
5  spring.jpa.hibernate.ddl-auto=none
6
7  # 로컬 JWT 시크릿
8  spring.jwt.secret=SahL5G83mGzruLJzffw-eD7RqphsXlUJPBwxu0ncBFd
```

application-prod.properties - 배포용

1	# 배포용
2	 spring.datasource.url=\${MYSQL_URL}
3	spring.datasource.username=\${MYSQL_USER}
4	spring.datasource.password=\${MYSQL_PASSWORD}
5	spring.jpa.hibernate.ddl-auto=update
6	

.github/workflows/deploy.yml

1	name: CI/CD Pipeline # GitHub Actions 워크플로우의 이름
2	
3	on:
4	push:
5	branches:
6	- main # main 브랜치에 push 이벤트가 발생했을 때 워크플로우 실행
7	
8	jobs:
9	build-and-deploy:
10	runs-on: ubuntu-latest # 최신 Ubuntu 환경에서 작업 실행
11	
12	steps:
13	- name: Checkout source
14	uses: actions/checkout@v3 # GitHub 저장소의 소스코드를 가져오는 액션
15	
16	- name: Set up JDK 17
17	uses: actions/setup-java@v3 # JDK 설정 액션
18	with:
19	java-version: '17' # Java 17 버전 사용
20	distribution: 'temurin' # AdoptOpenJDK의 Temurin 배포판 사용
21	
22	- name: mvnw 실행 권한 부여
23	run: chmod +x ./mvnw # mvnw 파일(Wrapper)에 실행 권한 부여
24	
25	- name: Maven 빌드 실행
26	run: ./mvnw clean package -DskipTests # Maven 빌드 실행 (테스트 생략)
27	# run: ./mvnw clean verify # 테스트 포함 빌드 시 이 명령어 사용
28	

```

29 - name: Docker Hub 로그인
30   uses: docker/login-action@v3 # Docker Hub에 로그인하는 액션
31   with:
32     username: ${ secrets.DOCKER_USERNAME } # GitHub에 저장된 시크릿 값 사용
33     password: ${ secrets.DOCKER_ACCESS_TOKEN }
34
35 - name: Docker 이미지 빌드
36   run: docker build -t ${ secrets.DOCKER_USERNAME }/jwt-app:latest . # 애플리케이션을 도커 이미지로 빌드
37
38 - name: Docker 이미지 푸시
39   run: docker push ${ secrets.DOCKER_USERNAME }/jwt-app:latest # 빌드한 이미지를 Docker Hub에 업로드
40
41 # .env 파일을 Actions 내에서 동적으로 생성
42 - name: .env 파일 생성
43   run: |
44     echo "SPRING_JWT_SECRET=${ secrets.SPRING_JWT_SECRET }" >> .env # Spring JWT 시크릿 키
45     echo "MYSQL_URL=${ secrets.MYSQL_URL }" >> .env # DB URL
46     echo "MYSQL_USER=${ secrets.MYSQL_USER }" >> .env # DB 사용자명
47     echo "MYSQL_PASSWORD=${ secrets.MYSQL_PASSWORD }" >> .env # DB 비밀번호
48     echo "DOCKER_USERNAME=${ secrets.DOCKER_USERNAME }" >> .env # 도커 허브 사용자명

```

```

50 - name: EC2에 docker-compose.yml + .env 전송
51   uses: appleboy/scp-action@v0.1.7 # EC2로 파일 전송하는 액션(SCP 방식)
52   with:
53     host: ${ secrets.EC2_HOST } # EC2 호스트 주소
54     username: ${ secrets.EC2_USERNAME } # EC2 사용자명 (보통 ubuntu)
55     key: ${ secrets.EC2_PRIVATE_KEY } # EC2 개인 키 (.pem 내용)
56     source: "docker-compose.yml,.env" # 전송할 파일 목록 (쉼표로 구분)
57     target: "~/cicd-docker" # EC2 내 대상 경로
58
59 - name: EC2에서 docker-compose 실행
60   uses: appleboy/ssh-action@v1.0.0 # EC2에 SSH 접속하여 명령 실행
61   with:
62     host: ${ secrets.EC2_HOST }
63     username: ${ secrets.EC2_USERNAME }
64     key: ${ secrets.EC2_PRIVATE_KEY }
65     script: |
66       cd ~/cicd-docker # 도커 파일이 위치한 경로로 이동
67       docker pull ${ secrets.DOCKER_USERNAME }/jwt-app:latest # 최신 이미지 가져오기
68       docker compose down # 기존 컨테이너 종료
69       docker compose up -d # 백그라운드로 새 컨테이너 실행

```

Spring Boot에서는 개발환경(dev)과 배포환경(prod)을 나눠서 설정파일을 관리할 수 있다.
이를 위해 **Spring 프로파일(profile)** 기능을 사용한다.

application.properties → 기본 설정

application-dev.properties → 개발용 DB 및 설정

application-prod.properties → 배포용 Docker/MySQL 설정

실행 시 환경에 따라 적절한 프로파일이 자동 적용

application.properties에 추가

```
properties
```

```
spring.profiles.active=dev
```

배포용(Docker 실행시)

docker-compose.yml에 다음 환경 변수 추가

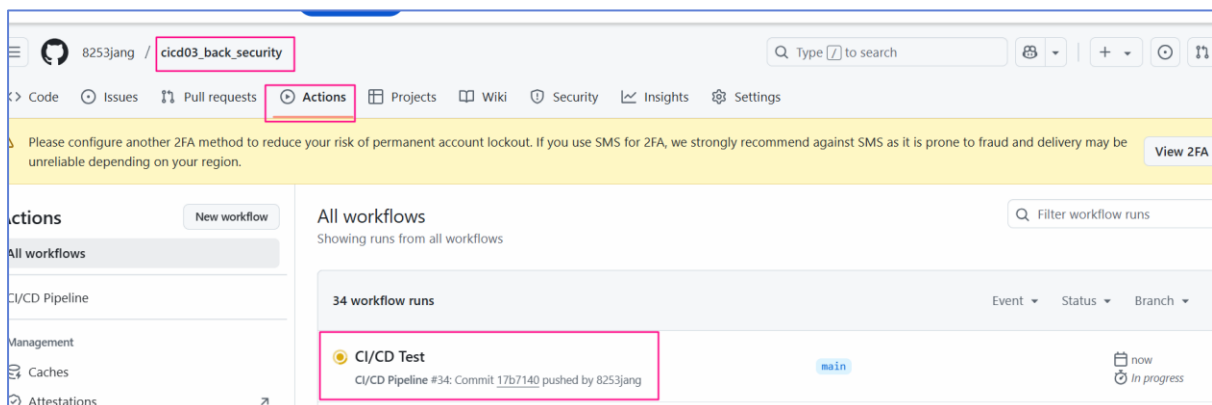
```
services:
  springboot-app:
    ...
    environment:
      ...
      SPRING_PROFILES_ACTIVE: prod
```

test해 보기

```
KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/back_security (main)
$ git add .

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/back_security (main)
$ git commit -m "CI/CD Test"
[main 17b7140] CI/CD Test
1 file changed, 1 insertion(+), 1 deletion(-)

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/back_security (main)
$ git push origin main
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 663 bytes | 663.00 KiB/s, done.
Total 9 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/8253jang/cicd03_back_security.git
b00c13c..17b7140 main -> main
```



← CI/CD Pipeline

✓ CI/CD Test #34

Summary

Jobs

✓ build-and-deploy

Run details

Usage

Workflow file

build-and-deploy

succeeded now in 57s

> ✓ Set up job

> ✓ Build appleboy/scp-action@v0.1.7

> ✓ Build appleboy/ssh-action@v1.0.0

> ✓ Checkout source

> ✓ Set up JDK 17

> ✓ mvnw 실행 권한 부여

> ✓ Maven 빌드 실행

> ✓ Docker Hub 로그인

> ✓ Docker 이미지 빌드

> ✓ Docker 이미지 푸시

> ✓ .env 파일 생성

> ✓ EC2에 docker-compose.yml + .env 전송

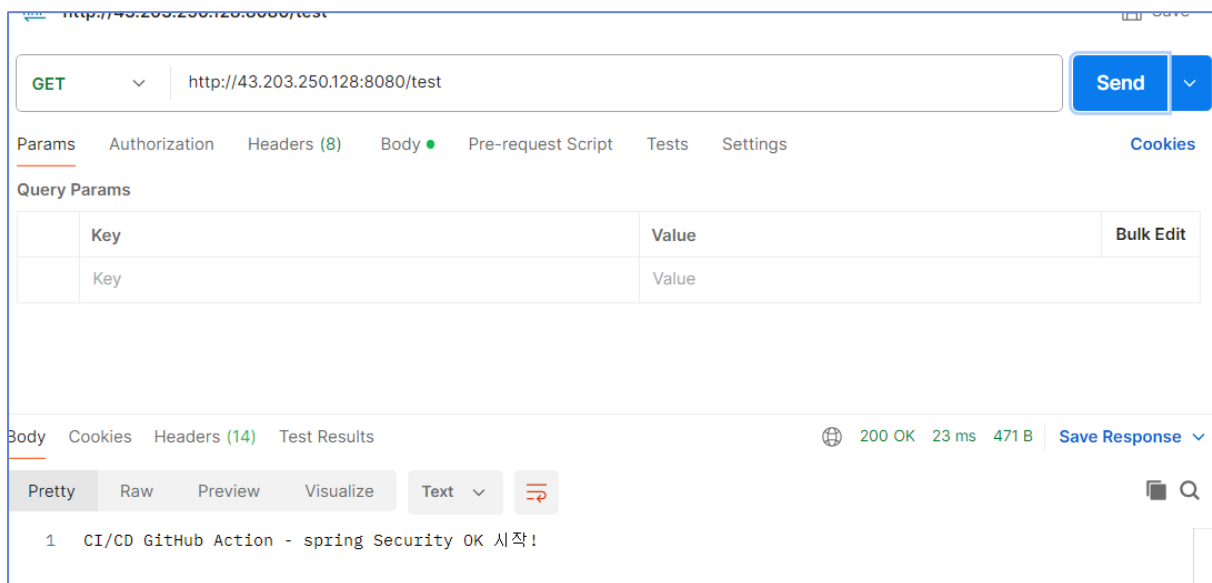
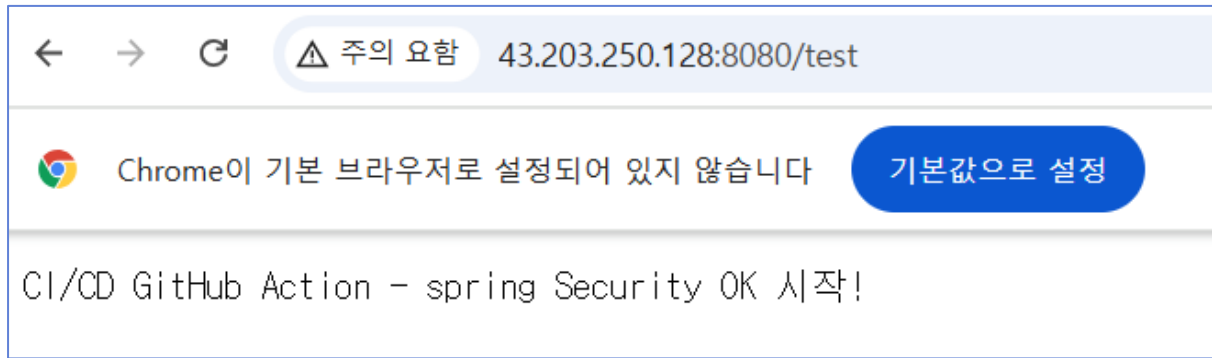
> ✓ EC2에서 docker-compose 실행

> ✓ Post Docker Hub 로그인

> ✓ Post Set up JDK 17

> ✓ Post Checkout source

> ✓ Complete job



Slack Webhook 사용하기

Slack Webhook은 외부 시스템이나 애플리케이션이 Slack 채널에 메시지를 보낼 수 있도록 해주는 HTTP 기반 인터페이스이다. 쉽게 말해, 특정 URL(웹주소)에 데이터를 보내면 Slack 채널에 메시지가 자동으로 전송되는 방식이다.

🔑 Slack Webhook의 핵심 개념	
항목	설명
Webhook URL	Slack이 발급해주는 고유 URL로, 여기에 HTTP POST 요청을 보내면 메시지가 Slack 채널로 전송됨
형식	일반적으로 JSON 형태로 메시지를 구성
용도	배포 알림, 에러 보고, 빌드 상태, 모니터링 경고 등을 Slack으로 자동 전송할 때 사용

Slack Webhook을 사용하는 이유

- 자동화: GitHub Actions, Jenkins, Docker 등에서 작업이 끝났을 때 자동으로 Slack 알림을 전송
- 팀 협업 강화: 중요한 작업의 상태를 팀원들이 실시간으로 확인 가능
- 에러 대응 속도 향상: 문제가 생겼을 때 Slack으로 즉시 알림을 받아 빠르게 대응 가능

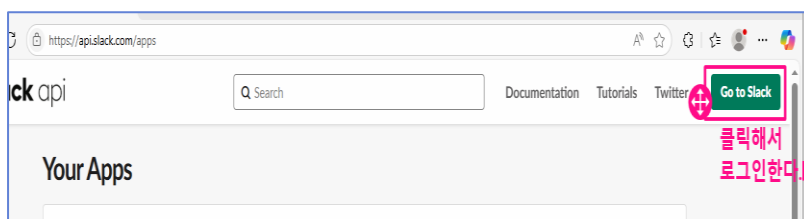
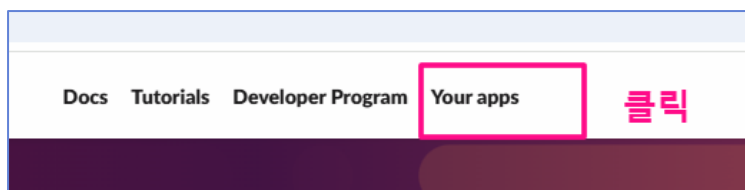
GitHub Actions의 **deploy.yml** 파일에서 Slack 알림을 추가하려면, Slack Webhook URL을 GitHub Secrets에 등록한 후 Slack 알림 전송 단계를 마지막에 넣으면 된다.

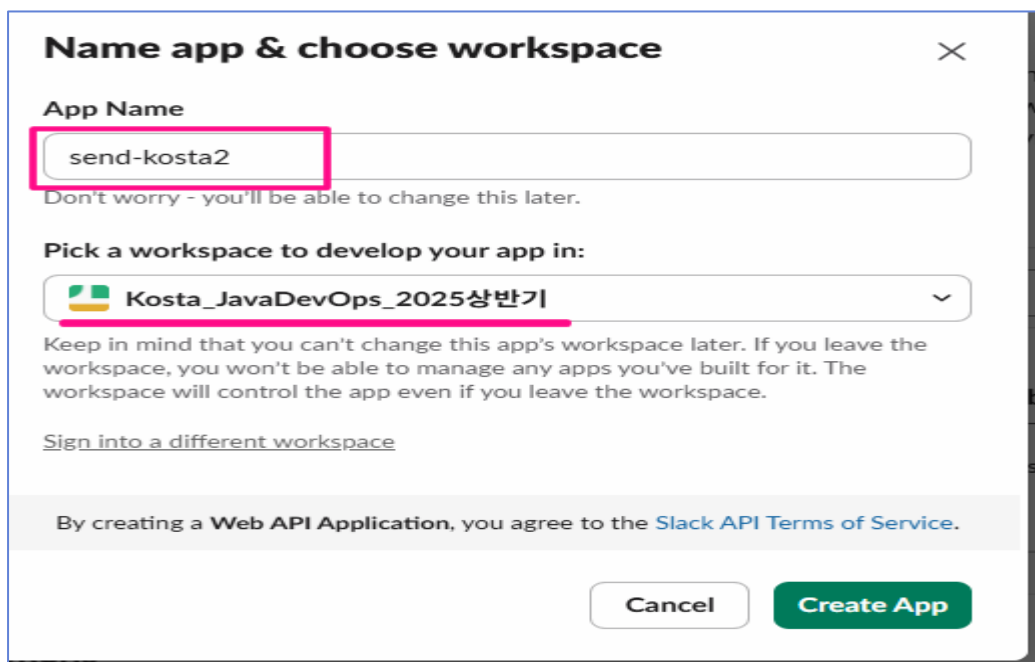
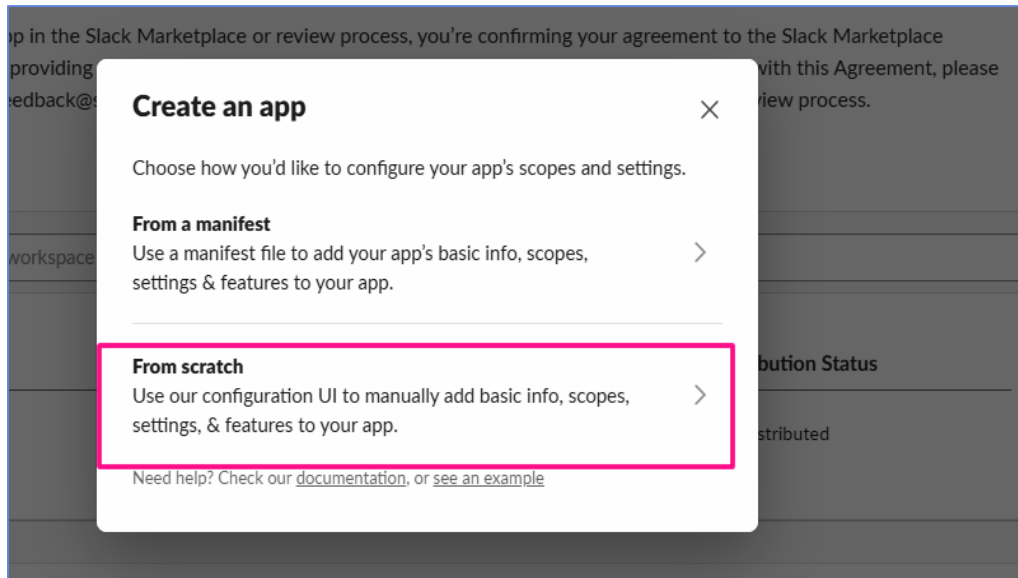
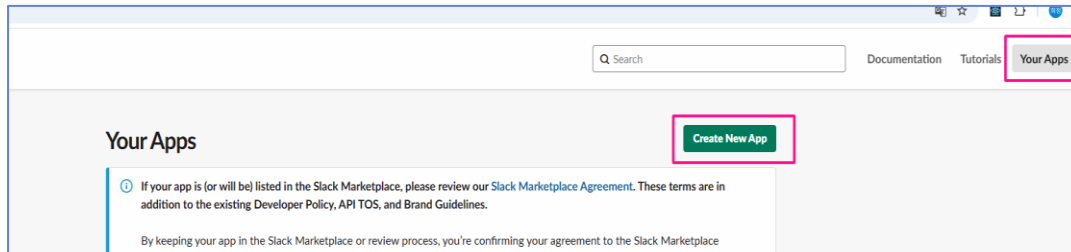
. Slack Webhook은 `{{ secrets.SLACK_WEBHOOK_URL }}`로 시크릿 처리한다.

Webhook URL 발급 방법

1. Slack에서 원하는 워크스페이스로 로그인
2. Incoming Webhooks 앱 설치
3. 사용할 채널 선택 후 "Webhook URL" 생성
4. 해당 URL을 GitHub Secrets 등 안전한 곳에 저장 후 사용

Slack 앱 설정 페이지(<https://api.slack.com>) 접속





send-kosta2

Settings

Basic Information

Collaborators

Socket Mode

Install App

Manage Distribution

Features

App Home

Agents & AI Apps NEW

Workflow Steps NEW

Org Level Apps

Incoming Webhooks

Interactivity & Shortcuts

Slash Commands

Steps from Apps LEGACY

OAuth & Permissions

Event Subscriptions

User ID Translation

App Manifest NEW

Beta Features

Submit to Slack Marketplace

Incoming Webhooks

Activate Incoming Webhooks

Incoming webhooks are a simple way to post messages from external sources into Slack. They make use of normal HTTP requests with a JSON payload, which includes the message and a few other optional details. You can include [message attachments](#) to display richly-formatted messages.

Adding incoming webhooks requires a bot user. If your app doesn't have a [bot user](#), we'll add one for you.

Each time your app is installed, a new Webhook URL will be generated.

If you deactivate incoming webhooks, new Webhook URLs will not be generated when your app is installed to your team. If you'd like to remove access to existing Webhook URLs, you will need to [Revoke All OAuth Tokens](#).

Webhook URLs for Your Workspace

To dispatch messages with your webhook URL, send your [message](#) in JSON as the body of an `application/json` POST request.

Add this webhook to your workspace below to activate this curl example.

Sample curl request to post to a channel:

```
curl -X POST -H 'Content-type: application/json' --data '{"text":"Hello, World!"}' YOUR_WEBHOOK_URL_HERE
```

Webhook URLs for Your Workspace

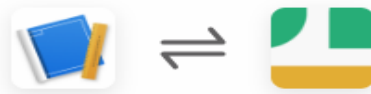
To dispatch messages with your webhook URL, send your [message](#) in JSON as the body of an `application/json` POST request.

Add this webhook to your workspace below to activate this curl example.

Sample curl request to post to a channel:

```
curl -X POST -H 'Content-type: application/json' --data '{"text":"Hello, World!"}' YOUR_WEBHOOK_URL_HERE
```

Webhook URL	Channel	Added By
No webhooks have been added yet.		
Add New Webhook		



send-kosta2에서 Kosta_JavaDevOps_2025상반기 Slack 워크스페이스에 액세스하기 위해 권한을 요청합니다.

send-kosta2은(는) 어디에 게시해야 할까요?

send-kosta2에는 앱으로 게시할 채널이 필요합니다.

공지사항

취소

허용

Webhook URLs for Your Workspace

To dispatch messages with your webhook URL, send your [message](#) in JSON as the body of an `application/json` POST request.

Add this webhook to your workspace below to activate this curl example.

Sample curl request to post to a channel:

```
curl -X POST -H 'Content-type: application/json' --data '{"text": "Hello, World!"}' https://hooks.slack.com/services/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
```

Copy

Webhook URL	Channel	Added By
https://hooks.slack.com/services/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	#공지사항	장희정강사 Jul 2, 2025

Copy

→ 복사해서 저장해둔다.
slack api들에 이용되는
webhook URL

Add New Webhook

GitHub Repository의 Settings > Secrets and variables > Actions에서 다음을 등록

SLACK_WEBHOOK_URL : Slack에서 발급받은 Webhook URL

.github > workflows > deploy.yml 수정

```
# 7. Slack 알림 전송
- name: Notify to Slack
  uses: rtCamp/action-slack-notify@v2
  env:
    SLACK_WEBHOOK: ${ secrets.SLACK_WEBHOOK_URL }
    SLACK_USERNAME: GitHub Actions
    SLACK_ICON_EMOJI: ':rocket:'
    SLACK_TITLE: '✅ EC2 배포 완료'
    SLACK_MESSAGE: 'Main 브랜치에 push됨 → EC2에 배포가 성공적으로 완료되었습니다! 🎉'
    SLACK_COLOR: good
```

아래 내용 추가(제공폴더 > ex06_springSecurity_mysql > docker_boot > deploy-slack.yml 참고)

```
# 7. Slack 알림 전송
- name: Notify to Slack
  uses: rtCamp/action-slack-notify@v2
  env:
    SLACK_WEBHOOK: ${ secrets.SLACK_WEBHOOK_URL }
    SLACK_USERNAME: GitHub Actions
    SLACK_ICON_EMOJI: ':rocket:'
    SLACK_TITLE: '✅ EC2 배포 완료'
    SLACK_MESSAGE: 'Main 브랜치에 push됨 → EC2에 배포가 성공적으로 완료되었습니다! 🎉'
    SLACK_COLOR: good
```

dplo.yml 수정 후 commit > push 해본다.

Slack에 아래와 같이 메시지 오는 것을 확인한다.


장희정강사 오후 3:00
 added an integration to this channel: [send-kosta](#)


send-kosta 앱 오후 3:07


8253jang

Ref	Event
refs/heads/main	push
Actions URL	Commit
CI/CD Pipeline	da877d


배포 성공

Main 브랜치에 push됨 → EC2에 성공적으로 배포되었습니다! 🎉

Powered By rtCamp's GitHub Actions Library | Triggered on this workflow run

 1
 

Slack 알림에 배포 상태(성공/실패)에 따라 분기처리

```

# Slack 알림 - 성공 시
- name: Notify to Slack on Success
  if: success()
  uses: rtCamp/action-slack-notify@v2
  env:
    SLACK_WEBHOOK: ${ secrets.SLACK_WEBHOOK_URL }
    SLACK_USERNAME: GitHub Actions
    SLACK_ICON_EMOJI: ':rocket:'
    SLACK_TITLE: '✅ 배포 성공'
    SLACK_MESSAGE: 'Main 브랜치에 push됨 → EC2에 성공적으로 배포되었습니다! 🚀'
    SLACK_COLOR: good

# Slack 알림 - 실패 시
- name: Notify to Slack on Failure
  if: failure()
  uses: rtCamp/action-slack-notify@v2
  env:
    SLACK_WEBHOOK: ${ secrets.SLACK_WEBHOOK_URL }
    SLACK_USERNAME: GitHub Actions
    SLACK_ICON_EMOJI: ':x:'
    SLACK_TITLE: '❌ 배포 실패'
    SLACK_MESSAGE: 'Main 브랜치에 push됨 → EC2 배포 중 오류가 발생했습니다. 🔥 확인이 필요합니다.'
    SLACK_COLOR: danger

```

if: success() → 앞 단계까지 모두 성공했을 때만 실행

if: failure() → 이전 단계 중 하나라도 실패했을 때 실행

아래 내용 추가(제공폴더 > ex06_springSecurity_mysql > docker_boot > deploy-slack02.yml 참고)

```

# Slack 알림 - 성공 시
- name: Notify to Slack on Success
  if: success()
  uses: rtCamp/action-slack-notify@v2
  env:
    SLACK_WEBHOOK: ${ secrets.SLACK_WEBHOOK_URL }
    SLACK_USERNAME: GitHub Actions
    SLACK_ICON_EMOJI: ':rocket:'
    SLACK_TITLE: '✅ 배포 성공'
    SLACK_MESSAGE: 'Main 브랜치에 push됨 → EC2에 성공적으로 배포되었습니다-추카추카! 🚀'
    SLACK_COLOR: good

# Slack 알림 - 실패 시
- name: Notify to Slack on Failure
  if: failure()
  uses: rtCamp/action-slack-notify@v2
  env:

```

```
SLACK_WEBHOOK: ${ secrets.SLACK_WEBHOOK_URL } # Slack에서 생성한 Webhook URL (GitHub Secrets로 관리)
SLACK_USERNAME: GitHub Actions # Slack에 표시될 발신자 이름
SLACK_ICON_EMOJI: ':x:' # Slack 메시지 앞에 표시될 아이콘 (실패용 이모지)
SLACK_TITLE: '❌ 배포 실패' # Slack 메시지의 제목 (굵은 텍스트로 표시됨)
SLACK_MESSAGE: 'Main 브랜치에 push됨 → EC2 배포 중 오류가 발생했습니다. 🚨 확인이 필요합니다.' # Slack 메시지의 본문 내용
(상세 메시지)
SLACK_COLOR: danger # 메시지 바 색상 (green=good, red=danger, yellow=warning)
```